

Faculty of Sciences Département of Computer Science



PREPARATORY WORK FOR THE MASTER THESIS

Exhaustive generation of non-isomorphic graphs

Auteur : Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2024–2025

Contents

1	Introduction	2
2	Preliminaries	4
2.1	Algebra Definitions	4
2.2	Graph Definitions	5
2.2.1	Definitions specifics to isomorphisms	6
2.3	Complexity Definitions	7
3	State of the Art	9
3.1	Isomorphisms Graph problem	9
4	Algorithm for the canonical graph	14
4.1	Search Tree and Node Invariant	14
4.1.1	Lemmas and corollaries	15
4.2	Implementation	16
4.3	Pruning	19

Chapter 1

Introduction

In graph theory, the graph isomorphism problem (GI) is a well-known challenge: determining whether two given graphs are isomorphic. Fully retracing its history is nearly impossible due to the vast number of works on the topic, but we will go over the most significant milestones. Throughout the 20th century, considerable research efforts were devoted to solving GI, with varying degrees of efficiency. One of the earliest and simplest approaches was backtracking — a widely known and relatively easy-to-implement technique, though it suffers from poor scalability and optimality [40]. Another method that emerged involved the construction of a so-called representative graph and a reordered graph to test for isomorphism [12]. Although the overall approach to solving the GI problem was later invalidated, it played a key role in stimulating deeper investigations into the problem.

Eventually, a more promising direction emerged: reducing graphs to their canonical form. The idea is straightforward — every graph can be transformed into a *unique* canonical representation [2], meaning that two isomorphic graphs will share the same canonical form. This shift in perspective turned GI from a problem of structural equivalence into one of equality: rather than directly checking isomorphism, we now simply compare canonical forms. This change led to more efficient solutions.

In 1981, Brendan D. McKay introduced a breakthrough with his software *Nauty* (“No Automorphisms, Yes?”) [32], which offered a much more efficient way of tackling GI. Initially written in Fortran and assembly, and later finalized in C, *Nauty* has undergone continuous optimization to maximise its speed and efficiency. The key idea behind *Nauty* is to compute a graph’s canonical form without relying on matrix representations. Instead, it uses a search tree combined with a partitioning system to generate the canonical form.

Nowadays, this canonical form capability opens the door to a powerful application: the exhaustive generation of non-isomorphic graphs. Given n vertices (and optionally, additional constraints), the goal is to enumerate all possible graphs that are non-isomorphic. Isomorph-free exhaustive generation is a problem that researchers have been working on for decades. For instance, the generation of cubic graphs has been studied since the 1960s, as these graphs are useful in various fields: in chemistry, they can model carbon networks; in mathematics and computer science, they often serve as minimal counterexamples in graph theory [9]. McKay, for his part, successfully connected his tool *Nauty* to the generation of non-isomorphic graphs. [31]

Nauty is much more than just a tool for canonical labelling and the generation of non-isomorphic graphs. It has evolved into a comprehensive program that supports a wide range of graph families and offers additional features, such as output graph statistics and other advanced analysis tools [34]. Furthermore, libraries based on *Nauty* have been developed for several programming languages—for example, Pynauty in Python [15], NautyGraphs in Julia [16] or even pl-nauty in Prolog. [18]

The goal of this Master’s thesis is to reimplement part of *Nauty* in a more modern programming language: Java. Recreating *Nauty* from scratch in Java will allow us to evaluate optimization differences, refresh its design, and extend its capabilities. For instance, the original *Nauty* does not support exhaustive generation or comparison for all possible graph families. In this updated version, we aim to include both some of the families already supported and additional ones, thereby contributing novel features to the tool.

As part of the preparatory work for this Master’s thesis, our focus will be on the canonical form of graphs: generating the unique representative graph for various types of input graphs. This represents the foundation for the first stage of the *Nauty*-inspired code — solving the Graph Isomorphism problem.

This document is structured as follows: first, we introduce the fundamentals and preliminaries required to fully understand the key concepts discussed in this thesis. We then present a comprehensive state-of-the-art section that summarizes previous research on the Graph Isomorphism problem. Finally, we explain how McKay’s canonical labelling algorithm works.

Chapter 2

Preliminaries

This chapter introduces all the definitions necessary to understand this document. For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [14].

2.1 Algebra Definitions

Definition 1 (informal). An *isomorphism* is a one-to-one correspondence (mapping) between two sets that preserves binary relationships between elements of the sets.

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has as identity element : 0, every element has an inverse element (each number with its opposite) and it's associative : for all numbers a, b, c in $\mathbb{R}$: $a + (b + c) = (a + b) + c$.

A *homomorphism* is a transformation of one set into another that preserves in the second set the relations between elements of the first. For example, given two groups, $(G, \cdot)$ and $(H, \circ)$, a *group homomorphism* from $(G, \cdot)$ to $(H, \circ)$ is a function $\phi : G \rightarrow H$ such that $\phi(g_1 \cdot g_2) = \phi(g_1) \circ \phi(g_2)$ for every $g_1, g_2 \in G$.

Let $(G, *)$ be a group with identity element e , and let X be a non-empty set. A *group action* of G on X exists if there is a map

$$\cdot : G \times X \rightarrow X, \quad (g, x) \mapsto g \cdot x$$

such that the following two properties hold:

1. Identity: $\forall x \in X, \quad e \cdot x = x,$
2. Compatibility: $\forall g, h \in G, \forall x \in X, \quad (g * h) \cdot x = g \cdot (h \cdot x).$

As an example, let S_n denote the *symmetric group* acting on a set X . We indicate the action of elements of S_n by exponentiation: for $x \in X$ and $g \in S_n$, x^g denotes the image of x under g . The same notation is used to indicate the induced action on complex structures derived from X . As additional notation, for $W \subseteq V$, we write $W^g = \{w^g : w \in W\}$.

Definition 3. An *equivalence relation*, denoted by $\sim$, on a set X is a binary relation that is reflexive, symmetric, and transitive.

For any element $x \in X$, the *equivalence class* of x under $\sim$ is the subset

$$[x] = \{y \in X \mid y \sim x\}.$$

The *quotient set* of X by $\sim$, denoted by

$$X/\sim = \{[x] \mid x \in X\},$$

is the set of all equivalence classes of X under $\sim$.

2.2 Graph Definitions

Definition 4. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V . The elements of V are the *vertices* of the graph G , the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the number of components of σ . If $\sigma = (v_1, v_2, \dots, v_k) \in V^*$ and $w \in V$ then $\sigma \parallel w$ denotes $(v_1, v_2, \dots, v_k, w)$. Furthermore, for $0 \leq s \leq k$, $[\sigma]_s := (v_1, v_2, \dots, v_s)$. The ordering $\leq$ on finite sequences is the lexicographic order: If $\sigma = (v_1, v_2, \dots, v_k)$ and $\sigma' = (v'_1, v'_2, \dots, v'_l)$, then $\sigma \leq \sigma'$ if σ is a prefix of σ' or there is some $j \leq \min\{k, l\}$ such that $v_i = v'_i$ for $i < j$ and $v_j < v'_j$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*.

Let $G = (V, E)$ be a graph. Let $\sim$ be an equivalence relation on V . The *quotient graph* of G with respect to $\sim$, denoted $G/\sim$, is a graph (V', E') such that :

1. $V' = V/\sim = \{[v] \mid v \in V\}$
2. $E' = \{[u][v] \mid uv \in E\}$

Definition 5. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \dots, n\}$). $\mathcal{G}$ represents the set of all graphs.

2.2.1 Definitions specifics to isomorphisms

Definition 6. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \dots, k\}$ for some k . The number of colours, i.e. k , is denoted by $|\pi|$. A cell of π is the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. We let Π be the set of all colourings.

If π, π' are colourings, then π' is *finer than or equal* to π , written $\pi' \preceq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

Here, the colouring is not related to a *proper colouring*, π is related to a *partitioning*.

A *discrete colouring* is a colouring in which each cell is a singleton, in which case $|\pi| = n$.

A *coloured graph* is a pair (G, π) where π is a colouring of G .

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 7. Let S_n denote the *symmetric group* acting on V .

There are several notations to consider:

1. if π is a colouring of V , then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\text{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}$; $\text{Aut}(G, \pi)$ is the *Automorphism group* of coloured graph (G, π) .

Definition 8. The *canonical form* is a function

$$C : \mathcal{G} \times \Pi \rightarrow \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

2.3 Complexity Definitions

For more informations on the complexity definitions, you can refer to Michael Sipser's book: Introduction to the Theory of Computation [44]

Definition 9 (minimalist). The class **NP** (comes from nondeterministic polynomial) is the set of decision problems for which the problem instances have certificates verifiable in polynomial time by a deterministic Turing machine, or alternatively the set of problems that can be solved in polynomial time by a nondeterministic Turing machine.

A decision problem is a member of **coNP** iff its complement is in the complexity class **NP**.

A decision problem B is **NP-complete** if it satisfies the two following conditions:

1. B must belong to the class **NP**.
2. Every problem in **NP** can be reduced to B in polynomial time.

Definition 10. A problem is **NP-intermediate** (**NPI**) if it is not **NP-complete** nor **P**.

Definition 11. **LOGSPACE** (or **L**) is the complexity class of decision problems that can be solved by a deterministic Turing machine using a logarithmic amount of writable memory.

Definition 12. An *oracle* for a language B is an external device that is capable of reporting whether any string w is a member of B . An oracle Turing machine is a modified Turing machine that has the additional capability of querying an oracle. We write M^B to describe an oracle Turing machine that has an oracle for language B .

The *Polynomial-Time Hierarchy* (**PH**) is a hierarchy of complexity classes that generalizes the classes **NP** and **coNP**. Formally, the base level of the hierarchy is defined as:

$$\Delta_0^P := \Sigma_0^P := \Pi_0^P := P,$$

where **P** denotes the class of decision problems solvable in polynomial time by a deterministic Turing machine. Then, for every $i \geq 0$, the hierarchy is defined recursively as follows:

- $\Delta_{i+1}^P := P^{\Sigma_i^P}$
- $\Sigma_{i+1}^P := NP^{\Sigma_i^P}$
- $\Pi_{i+1}^P := coNP^{\Sigma_i^P}$

Here, P^A denotes the class of problems solvable in polynomial time by a deterministic Turing machine equipped with an oracle for some complete problem in class A . The classes NP^A and $coNP^A$ are defined analogously, using nondeterministic and co-nondeterministic Turing machines, respectively.

For example, $\Sigma_i^P = NP$, $\Pi_i^P = coNP$, and $\Delta_2^P := P^{NP}$ is the class of problems solvable in polynomial time with access to an oracle for an NP -complete problem.

This definition implies that each level of the hierarchy is contained within the next. That is:

- $\Sigma_i^P \subseteq \Delta_{i+1}^P \subseteq \Sigma_{i+1}^P$
- $\Pi_i^P \subseteq \Delta_{i+1}^P \subseteq \Pi_{i+1}^P$

The union of all classes in this hierarchy forms the polynomial-time hierarchy, denoted by PH .

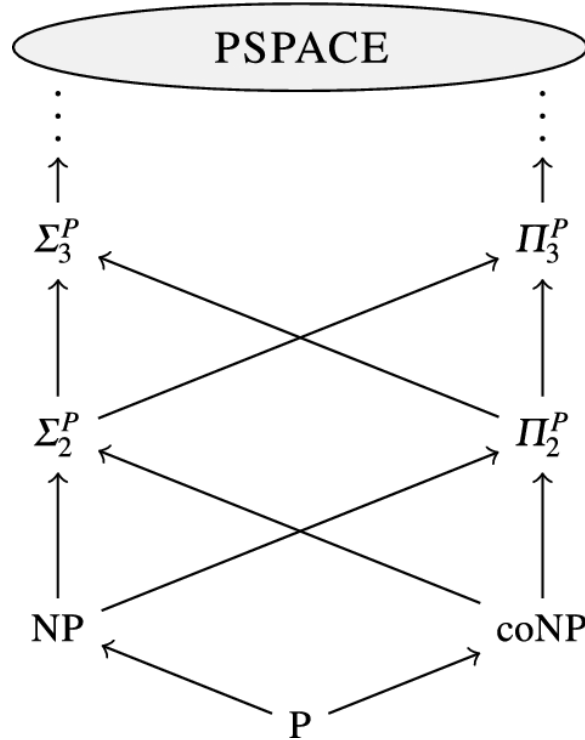


Figure 2.1: Polynomial Time Hierarchy where the arrows denote the inclusion: image from [6]

Chapter 3

State of the Art

3.1 Isomorphisms Graph problem

The Graph Isomorphism (GI) problem is very well known among computer scientists and mathematicians. It has been such a “star” target for algorithm designers that it was even referred to as a disease by researchers. [39]. GI belongs to the complexity class NP (cf. Definition 9). However, it is still unknown whether a polynomial-time algorithm exists to solve it — in other words, it is not known whether GI belongs to the complexity class P. Moreover, it is not known to be NP-complete. GI was referred to as an “open problem” in the 1970s; a problem of which it was not known whether they were NP-complete or in P (or neither) [19]. Actually, its NP-completeness is considered highly unlikely, as proving it NP-complete would imply a collapse of the polynomial-time hierarchy (PH) to its second level [20, 41]—a result that would shake the foundations of computational complexity theory. This is especially problematic because it is widely conjectured that all levels of PH are distinct and if GI were NP-complete, it would imply that all levels above the second would be equal to the second level.

For three decades, the fastest proven running time for the Graph Isomorphism problem was $e^{\mathcal{O}(\sqrt{n \log n})}$ [5]. But in 2016, it was shown that GI can be solved in quasi-polynomial time: $e^{\log(n)^{\mathcal{O}(1)}}$ [3]. Which makes one think even more that GI is only NP-intermediate and not NP-complete otherwise every problem in NP could be solved in quasi-polynomial time.

Nonetheless, We already known some polynomial-time algorithms for GI for specific graph families. These are often graphs with certain structural constraints, such as having a bounded Hadwiger number, a forbidden minor (see [38] in Russian, as cited in [33]), or an excluded topological subgraph [21]. Concrete examples of such graph classes include: graphs of bounded degree [30], bounded tree-width [8], and bounded eigenvalue multiplicities [4],

among others. The case of graphs with bounded genus was also addressed in earlier work [17]; however, the validity of the proposed polynomial-time algorithm was questioned later challenged [35].

For some of the graph classes mentioned, we can go even further. GI restricted to planar graphs and interval graphs can be solved in LOGSPACE (c.f. Definition 11), which represents a significant gain in terms of memory usage [13, 28].

Unfortunately, most of these polynomial-time algorithms are theoretical and hard to implement in practice, with the exception of some well-structured families like trees [1], planar graphs [24], and interval graphs [11].

Moreover, GI restricted to certain graphs do not change the difficulty of the problem; they are as hard as the general problem. Comme exemple de ces graphes, nous pouvons citer les graphes bipartis, chordal graphs, graphs of bounded degeneracy or even graphs of bounded expansion [22].

To solve the Graph Isomorphism (GI) problem, three main types of algorithms must be considered [22]:

- *Graph theoretic algorithms*: Classical algorithms that exploit structural properties of specific graph classes.
- *Group theoretic algorithms*: Based on computing generators for the automorphism group of a graph. These methods leverage group theoretic properties and have dominated GI research since the early 1980s.
- *Combinatorial algorithms*: Simple and general purpose algorithms that do not rely on the specific structure of graph classes.

All combinatorial algorithms are based on the idea of *colour refinement* (the 1-dimensional version of the Weisfeiler-Leman algorithm [26]), which aims to compute a *stable colouring* of a graph.

To use colour refinement as an isomorphism test, we apply it to the disjoint union of the input graphs G and G' . The algorithm *distinguishes* G and G' if there exists a colour c in the stable colouring such that G and G' contain different numbers of vertices of colour c . Colour refinement *identifies* a graph G if it distinguishes G from all non-isomorphic graphs; that is, no non-isomorphic graph shares the same stable colouring as G . However, if two graphs share the same stable partition, this is not a sufficient condition to prove that they are isomorphic; it is only a necessary one. Therefore, colour refinement does not fully solve the Graph Isomorphism problem, but rather provides a useful indication.

Colour refinement is not only used in the context of Graph Isomorphism; it also plays a role in reducing the cost of solving linear programs [23], as well as in computing graph kernels [43].

Algorithm 1 Example of Colour Refinement Algorithm [22]

```
1: Initialisation: Assign the same initial colour to all vertices.
2: repeat
3:   Refinement Step:
4:   for all pairs of vertices  $v, w \in V$  do
5:     for all colours  $c$  in the current colouring do
6:       if the number of neighbours of  $v$  of colour  $c$  differs from that
         of  $w$  then
7:         Assign different colours to  $v$  and  $w$ 
8:       end if
9:     end for
10:  end for
11: until (the colouring is stable)
```

Regarding the time complexity, for a graph G with n vertices and m edges, a stable colouring can be computed in time $\mathcal{O}((n + m) \log n)$ [10], and has a lower bound of $\Omega((n + m) \log n)$ for the natural class of iterative partition refinement algorithms [7]. This gives a tight bound of $\Theta((n + m) \log n)$ for such algorithms.

Now that the concept of colour refinement is well defined, we can turn to the types of combinatorial algorithms that build upon it:

- *Weisfeiler–Leman algorithm (or k -WL)* [26]: This is a generalization of the colour refinement algorithm. The principle remains largely the same, but instead of comparing individual nodes, the algorithm works on k -tuples of vertices. The central concept introduced is that of *i -neighbours*. For $\vec{v}, \vec{w} \in V^k$ and for $i \in \{1, \dots, k\}$, the tuples $\vec{v}$ and $\vec{w}$ are said to be *i -neighbours* if $v_j = w_j$ for all $j \neq i$.

For a graph G with n vertices, the stable colouring computed by k -WL can be obtained in time $\mathcal{O}(n^k \log n)$.

- *Individualisation-Refinement*: This approach consists of fixing vertices (individualisation) combined with the refinement of partitions of the vertex set. It was first introduced by Parris and Read [36], as cited in [33]). The first implementation of this technique appeared in *Nauty*. The key idea in *Nauty* is to construct a *search tree*, where individualisation and refinement are applied recursively to guide the traversal. Once the search tree is fully constructed, a *node invariant* function is applied in order to compute a *canonical labelling* of the graph; a unique representation (or canonical form) of the graph. This canonical labelling is central to solving the graph isomorphism problem (GI): two graphs are isomorphic if and only if their canonical forms are identical.

Algorithm 2 k -dimensional Weisfeiler-Leman (k-WL) algorithm

Require: Graph $G = (V, E)$, integer $k \geq 1$

Ensure: Stable colouring of V^k

```
1: Initialization:
2: for all tuples  $\vec{v}, \vec{w} \in V^k$  do
3:   if  $\vec{v} \not\cong \vec{w}$  then  $\triangleright$  No isomorphism between induced subgraphs
4:     Assign different initial colours to  $\vec{v}$  and  $\vec{w}$ 
5:   end if
6: end for
7: repeat
8:   Refinement step:
9:   for all tuples  $\vec{v}, \vec{w} \in V^k$  do
10:    for  $i = 1$  to  $k$  do
11:      Compare multisets of colours of  $i$ -neighbours of  $\vec{v}$  and  $\vec{w}$ 
12:      if they differ then
13:        Assign different colours to  $\vec{v}$  and  $\vec{w}$ 
14:      end if
15:    end for
16:  end for
17: until the colouring is stable
```

Over time, several other GI solvers based on the individualisation-refinement Sparadigm have emerged:

- *Saucy*, introduced in 2004, has the same idea than *Nauty* but was designed to outperform it on sparse graphs [13].
- A few years later, *Bliss* was released. It also computes canonical forms but incorporates additional techniques that enhance performance on more complex graphs ([25], as cited in [33]).
- *Conauto*, introduced in 2009, has shown remarkable efficiency on random graphs and certain families of hard instances [29].
- Finally, *Traces* was developed [37], a tool that can compete with *Nauty* in terms of performance on the Graph Isomorphism problem. Today, *Traces* and *Nauty* are tightly integrated and distributed together [33].

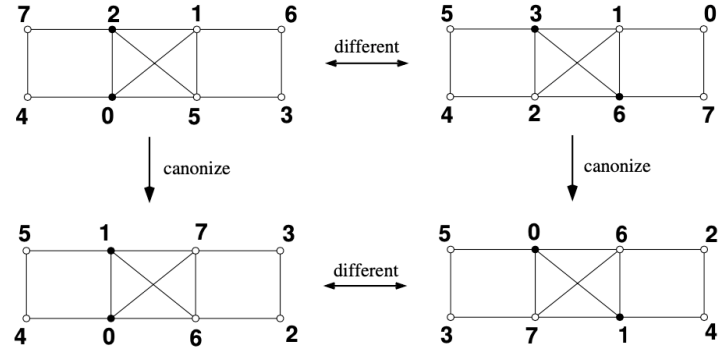


Figure 3.1: Example of GI with canonical labelling from [34]

Finally, An alternative approach to solving GI involves using linear programming. Given the adjacency matrices of two graphs G and H , they are isomorphic if and only if there exists a permutation matrix X such that $X^\top AX = B$.

This formulation can be relaxed using LP techniques, notably the Sherali–Adams hierarchy. In particular, level- k Sherali–Adams relaxations offer a powerful framework that has been investigated for graph isomorphism testing [22].

Chapter 4

Algorithm for the canonical graph

Now that the state of the art has been established, we can turn our attention to the core of this work. This chapter focuses primarily on McKay's contribution to the construction of a canonical form of a graph [33].

4.1 Search Tree and Node Invariant

The foundation of the algorithm is a very specific kind of tree: a *search tree* (c.f. [Definition 4](#)). Each node in this tree corresponds to a sequence of vertices of a coloured graph (see [Definition 6](#)) derived from the original graph. The root of the tree is the empty sequence, and as we move down the tree, the sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a "controlled" manner, the colouring of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete.

To formally define this newly constructed tree, we begin by introducing the *refinement function*, which will be used to specify the colouring associated with a given sequence.

Definition 13. The *Refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \rightarrow \Pi$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$,

1. $R(G, \pi, v) \preceq \pi$.
2. if $v \in \sigma$, then $\{v\}$ is a cell of $R(G, \pi, \sigma)$.

3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

Finally, to complete the formal definition of our newly constructed tree, we need to specify how the children of each node are generated. A child is created by selecting a non-singleton cell from the current colouring (called the target cell) and adding one of its elements to the sequence.

We will define the function called the *target cell selector*, which is responsible for choosing a non-singleton cell from the colouring (if such a cell exists).

Definition 14. The *Target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \rightarrow 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$

1. if $R(G, \pi_0, \sigma)$ is discrete, then $T(G, \pi_0, \sigma) = \emptyset$.
2. if $R(G, \pi_0, \sigma)$ is not discrete, then $T(G, \pi_0, \sigma)$ is a non-singleton cell of $R(G, \pi_0, \sigma)$.
3. for any $g \in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0) . All the nodes of the tree are elements of V^* . This search tree has some characteristics to keep in mind:

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence $()$.
- if σ is a node of $\mathcal{T}(G, \pi_0)$, let $W = T(G, \pi_0, \sigma)$. Then the children of π are $\{\sigma \parallel w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

4.1.1 Lemmas and corollaries

To conclude the construction of the tree, there are a few lemmas and corollaries to introduce, which will be very helpful for the upcoming definitions.

Lemma 1. For any $G \in \mathcal{G}, \pi_0 \in \Pi, g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.

Corollary 2. Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in \text{Aut}(G, \pi_0)$. Then σ^g , is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.

Lemma 3. Let σ be a node of $(\mathcal{G}, \pi_i)$ and let $\pi = R(G, \pi_0, \sigma)$. Then $\text{Aut}(G, \pi)$ is the point-wise stabilizer of σ in $\text{Aut}(G, \pi_0)$.

Now that we have our base search tree $\mathcal{T}(G, \pi_0)$, we can examine how it can be used to compute the canonical form of the original graph and $\text{Aut}(G, \pi_0)$. (c.f [Definition 7](#)).

Definition 15. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \rightarrow \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$.

The function ϕ has a few properties to take into account:

1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$,
then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma'_1 \in \mathcal{T}(G, \pi_0, \sigma')$ we have
 $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma'_1)$.
2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete,
then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^\pi = G^{\pi'}$ (equality relation).
3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.
4. Leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. This is the case, if there is a unique $g \in \text{Aut}(G, \pi_0)$ such that $\sigma^g = \sigma'$, with
 $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$.

By [Corollary 2](#), if σ is a leaf of $\mathcal{T}(G, \pi_0)$, then so is σ^g for any $g \in \text{Aut}(G, \pi_0)$. Moreover, due to the properties of ϕ , all such leaves σ^g share the same ϕ -value, and no other leaf has this value. Consequently, for each leaf σ , we can say that:

$$\begin{aligned} \text{Aut}(G, \pi_0) = \{ R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1} : \sigma' \text{ is a leaf of } \mathcal{T}(G, \pi_0) \\ \text{with } \phi(G, \pi_0, \sigma') = \phi(G, \pi_0, \sigma) \}. \end{aligned}$$

Finally, we can define the canonical form:

$$\phi^*(G, \pi_0) = \max \{ \phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0) \},$$

4.2 Implementation

We now have all the theoretical foundations necessary to understand the construction of the search tree. We can proceed to put this knowledge into practice by describing the generic algorithm used to compute both the canonical form and the automorphism group of a graph.

In order to ensure that the properties of the [refinement function](#) defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer) than the original colouring. To this end, we rely on a theorem: for every colouring π , there exists a unique (up to cell ordering) coarsest equitable colouring π' such that $\pi' \preceq \pi$.

Therefore, we need to define additional helper functions in order to obtain a practical implementation of the refinement function.

Definition 16. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [33])

```

1: Data:  $\pi$  is the input colouring and  $\alpha$  is a sequence of some cells of  $\pi$ 
2: Result: the final value of  $\pi$  is the output colouring
3: while  $\alpha$  is not empty and  $\pi$  is not discrete do
4:   Remove some element  $W$  from  $\alpha$ 
5:   for each cell  $X$  of  $\pi$  do
6:     Let  $X_1, \dots, X_k$  be the fragments of  $X$  distinguished according to
       the number of edges from each vertex to  $W$ 
7:     Replace  $X$  by  $X_1, \dots, X_k$  in  $\pi$ 
8:     if  $X \in \alpha$  then
9:       Replace  $X$  by  $X_1, \dots, X_k$  in  $\alpha$ 
10:    else
11:      Add all but one of the largest of  $X_1, \dots, X_k$  to  $\alpha$ 
12:    end if
13:  end for
14: end while

```

Definition 17. The *individualized function* $I : \Pi \times V \rightarrow \Pi$

such that, if v is a vertex in a non-singleton cell of π and $\pi' = I(\pi, v)$, then for $w \in V$,

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v; \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

We say that the vertex v is individualized in π . We see that $I(\pi, v)$ differs from π in that a unique colour has been given to vertex v . Now we can define a refinement function. For a sequence of vertices $v_1, v_2, \dots$, define

- $R(G, \pi_0, ()) = F(G, \pi_0, \text{ a list of all the cells of } \pi_0)$
- $R(G, \pi_0, (v_1)) = F(G, I((R(G, \pi_0, ())), v_1), \{v_1\})$
- $R(G, \pi_0, (v_1, v_2)) = F(G, I((R(G, \pi_0, (v_1))), v_2), \{v_2\})$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G, I((R(G, \pi_0, (v_1, v_2))), v_3), \{v_3\})$
- and so on.

The algorithm satisfies the required properties of a refinement function and performs well in practice. However, the refinement step is the most compu-

tationally expensive part of the algorithm, which is why special care must be taken in its implementation.

For the implementation of the **target cell selection function** in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilizes the current sequence [32]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [27].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

Now that the search tree is implemented, we need to define the **node invariant function**, which will allow us to determine the automorphism group and the canonical form. To compute this function, we can rely on two related sources:

1. For each node σ , there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.
2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \dots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph* $Q(G, \pi)$ (see **Definition 4**), assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π , and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possi-

bly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2 . The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [v]_i))$ for $i = 0, \dots, |v|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

4.3 Pruning

To avoid generating unnecessary information in our search tree, which would result in an overly large tree, we aim to construct the most optimized tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define three types of pruning strategies to optimize our construction:

1. Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$. Operation $P_A(\sigma, \sigma')$ is to remove $T(G, \pi_0, \sigma')$.
2. Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$. Operation $P_B(\sigma, \sigma')$ is to remove $T(G, \pi_0, \sigma')$.
3. Suppose $g \in \text{Aut}(G, \pi_0)$ and suppose $\sigma < \sigma'$ are nodes of $T(G, \pi_0)$ such that $\sigma^g = \sigma'$. Operation $P_C(\sigma, g)$ is to remove $T(G, \pi_0, \sigma')$.

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.

Theorem 4. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $P_A(\sigma, \sigma')$ or $P_C(\sigma, g)$ are performed. Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.*
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $P_B(\sigma, \sigma')$ $P_C(\sigma'', g)$ are performed, where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$. Let $g_1, \dots, g_k$ be the automorphisms used in the operations P_C that were performed, and let $A = \{g \in \text{Aut}(G, \pi_0) : v_0^g \text{ is a remaining leaf}\}$. Then $\text{Aut}(G, \pi_0)$ is generated by $\{g_1, \dots, g_k\} \cup A$.*

We can put in practice our different pruning strategies. *Nauty* will generate its tree in depth-first order. The lexicographically least leaf σ_1 is kept and if the canonical labelling is sought (rather than just the automorphism group), the leaf σ^* with the greatest invariant discovered so far is also kept. A discrete node σ is pruned if neither $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, [\sigma_1]_{|\sigma|})$ or $\phi(G, \pi_0, \sigma) \geq \phi(G, \pi_0, [\sigma^*]_{|\sigma|})$. Such operations have both type $P_A(\sigma^*, \sigma)$

and $P_V(\sigma_1*, \sigma)$. Automorphisms are found by discovering leaves equivalent to σ_1 or σ^* , and also to a limited extent from the properties of equitable colourings. Pruning operation P_C is performed wherever possible, as high in the tree as possible (i.e., at the children of the nearest common ancestor of the two leaves found to be equivalent). Until a recent version of *Nauty*, the only automorphisms used for pruning operation P_C were those directly discovered, without any attempt to compose them. Now *Nauty* use the random Schreier method [42] to perform more complete pruning.

An apparent disadvantage of breadth-first order is that pruning by automorphisms (operation P_C) is only possible when automorphisms are known, which in general requires leaves of the tree. To remedy this problem, for every node a single path, called an “experimental path”, is generated from that node down to a leaf of the tree. Automorphisms are found by comparing the labelled graphs that correspond to those leaves, with the value of $\phi(G, \pi_0, \sigma)$ at the leaf being used to avoid most unnecessary comparisons. We have found experimentally that generating experimental paths randomly tends to find automorphisms that generate larger subgroups, so that the group requires fewer generators altogether and more of the group is available early for pruning. The group generated by the automorphisms found so far is maintained using the random Schreier method.

Bibliography

- [1] A. V. Aho and J. E. Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.
- [2] V. L. Arlazarov, I. I. Zuev, A. V. Uskov, and I. Faradzhev. An algorithm for the reduction of finite non-oriented graphs to canonical form. *USSR Computational Mathematics and Mathematical Physics*, 14(3), 1974.
- [3] L. Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 684–697, 2016.
- [4] L. Babai, D. Y. Grigoryev, and D. M. Mount. Isomorphism of graphs with bounded eigenvalue multiplicity. In *Proceedings of the fourteenth annual ACM symposium on Theory of computing*, pages 310–324, 1982.
- [5] L. Babai, W. M. Kantor, and E. M. Luks. Computational complexity and the classification of finite simple groups. In *24th Annual Symposium on Foundations of Computer Science (Sfcs 1983)*, pages 162–171. IEEE, 1983.
- [6] S. Baruah. Feasibility analysis for hpc-dag tasks. *Real-Time Systems*, 58:1–19, June 2022. DOI: [10.1007/s11241-022-09380-z](https://doi.org/10.1007/s11241-022-09380-z).
- [7] C. Berkholz, P. Bonsma, and M. Grohe. Tight lower and upper bounds for the complexity of canonical colour refinement. *Theory of Computing Systems*, 60:581–614, 2017.
- [8] H. L. Bodlaender. Polynomial algorithms for graph isomorphism and chromatic index on partial k -trees. *Journal of Algorithms*, 11(4):631–643, 1990.
- [9] G. Brinkmann, J. Goedgebeur, and N. Van Cleemput. The history of the generation of cubic graphs. *International Journal of Chemical Modeling*, 5(2-3):203–225, 2013.
- [10] A. Cardon and M. Crochemore. Partitioning a graph in $\mathcal{O}(|A| \log_2 |V|)$. *Theoretical Computer Science*, 19(1):85–98, 1982.

- [11] C. J. Colbourn and K. S. Booth. Linear time automorphism algorithms for trees, interval graphs, and planar graphs. *SIAM Journal on Computing*, 10(1):203–225, 1981.
- [12] D. G. Corneil and C. C. Gotlieb. An efficient algorithm for graph isomorphism. *Journal of the ACM (JACM)*, 17(1):51–64, 1970.
- [13] P. T. Darga, M. H. Liffiton, K. A. Sakallah, and I. L. Markov. Exploiting structure in symmetry detection for CNF. In *Proceedings of the 41st Annual Design Automation Conference*, pages 530–534, 2004.
- [14] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.
- [15] P. Dobsan. Pynauty: a python interface for nauty, 2023. URL: <https://github.com/pdobsan/pynauty>. Accessed: 2025-05-11.
- [16] T. Doe and contributors. Nautygraphs.jl: a julia interface for nauty, 2023. URL: <https://discourse.julialang.org/t/new-nautygraphs-jl/119029>. Accessed: 2025-05-11.
- [17] I. S. Filotti and J. N. Mayer. A polynomial-time algorithm for determining the isomorphism of graphs of fixed genus. In *Proceedings of the twelfth annual ACM symposium on Theory of computing*, pages 236–243, 1980.
- [18] M. Frank and M. Codish. Logic programming with graph automorphism: integrating nauty with prolog (tool description). *Theory and Practice of Logic Programming*, 16(5-6):688–702, 2016.
- [19] M. R. Garey and D. S. Johnson. *Computers and intractability*, volume 29. wh freeman New York, 2002.
- [20] O. Goldreich, S. Micali, and A. Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM (JACM)*, 38(3):690–728, 1991.
- [21] M. Grohe. *Structural and logical approaches to the graph isomorphism problem*. In *Proceedings of the 2012 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 188–188. DOI: [10.1137/1.9781611973099.16](https://doi.org/10.1137/1.9781611973099.16). URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611973099.16>.
- [22] M. Grohe. Structural and logical approaches to the graph isomorphism problem. In *SODA*, page 188, 2012.
- [23] M. Grohe, K. Kersting, M. Mladenov, and E. Selman. Dimension reduction via colour refinement. In *Algorithms-ESA 2014: 22th Annual European Symposium, Wroclaw, Poland, September 8-10, 2014. Proceedings 21*, pages 505–516. Springer, 2014.

- [24] J. E. Hopcroft and J. K. Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*, STOC '74, pages 172–184, Seattle, Washington, USA. Association for Computing Machinery, 1974. ISBN: 9781450374231. DOI: [10.1145/800119.803896](https://doi.org/10.1145/800119.803896). URL: <https://doi.org/10.1145/800119.803896>.
- [25] T. Junttila and P. Kaski. Conflict propagation and component recursion for canonical labeling. In *International Conference on Theory and Practice of Algorithms in (Computer) Systems*, pages 151–162. Springer, 2011.
- [26] S. Kiefer. *Power and limits of the Weisfeiler-Leman algorithm*. PhD thesis, Dissertation, RWTH Aachen University, 2020, 2020.
- [27] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.
- [28] J. Köbler, S. Kuhnert, B. Laubner, and O. Verbitsky. Interval graphs: canonical representations in logspace. *SIAM Journal on Computing*, 40(5):1292–1315, 2011.
- [29] J. L. López-Presa, A. F. Anta, and L. N. Chiroque. Conauto-2.0: fast isomorphism testing and automorphism group computation. *arXiv preprint arXiv:1108.1060*, 2011.
- [30] E. M. Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *Journal of computer and system sciences*, 25(1):42–65, 1982.
- [31] B. D. McKay. Isomorph-free exhaustive generation. *Journal of Algorithms*, 26(2):306–324, 1998. ISSN: 0196-6774. DOI: <https://doi.org/10.1006/jagm.1997.0898>. URL: <https://www.sciencedirect.com/science/article/pii/S0196677497908981>.
- [32] B. D. McKay et al. Practical graph isomorphism, 1981.
- [33] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.
- [34] B. D. McKay. *nauty and Traces User's Guide (Version 2.8.9)*. <https://users.cecs.anu.edu.au/~bdm/nauty/nug28.pdf>. Australian National University. Canberra, Australia, Aug. 2024.
- [35] W. Myrvold and W. Kocay. Errors in graph embedding algorithms. *Journal of Computer and System Sciences*, 77(2):430–438, 2011.
- [36] R. Parris and R. Read. A coding procedure for graphs. *Scientific Report. UWI/CC*, 10, 1969.
- [37] A. Piperno. Search space contraction in canonical labeling of graphs. *arXiv preprint arXiv:0804.4881*, 2008.

- [38] I. N. Ponomarenko. Isomorphism problem for classes of graphs closed under contractions. *Zapiski Nauchnykh Seminarov POMI*, 174:147–177, 1988.
- [39] R. C. Read and D. G. Corneil. The graph isomorphism disease. *Journal of graph theory*, 1(4):339–363, 1977.
- [40] D. C. Schmidt and L. E. Druffel. A fast backtracking algorithm to test directed graphs for isomorphism using distance matrices. *Journal of the ACM (JACM)*, 23(3):433–445, 1976.
- [41] U. Schöning. Graph isomorphism is in the low hierarchy. *Journal of Computer and System Sciences*, 37(3):312–323, 1988.
- [42] Á. Seress. *Permutation group algorithms*, number 152. Cambridge University Press, 2003.
- [43] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9), 2011.
- [44] M. Sipser. Introduction to the theory of computation. *ACM Sigact News*, 27(1):27–29, 1996.